

SISTEMA MÓVIL BUMAND

Fundación Diaconía FRIF-IFD

MANUAL EXHAUSTIVO DE ARQUITECTURA, COMPONENTES Y SISTEMA OPERATIVO MÓVIL

Clean Architecture, Gestión Reactiva con Riverpod, Radar Geofence a 60 FPS,
Captura Geoespacial de Tramos y Suite de 39 Pruebas Automatizadas al 100 %

Módulo del Repositorio:	bumand-movil
Framework y Lenguaje:	Flutter 3.29+ / Dart 3.7+
Patrón Arquitectónico:	Clean Architecture por Dominios y Funcionalidad
Gestión de Estado:	Flutter Riverpod (NotifierProvider, AsyncNotifier)
Sensores Nativos:	Geolocator (Fórmula de Haversine y Anti-Spoofing)
Almacenamiento Local:	SharedPreferences Criptoseguro
Calidad de Software:	39 Pruebas Unitarias y de Widgets Aprobadas (100 %)
Gestión Académica:	2026

Índice

1. Fundamentación y Contexto Operativo	2
2. Arquitectura de Software y Principios de Diseño	2
2.1. Estratificación de Responsabilidades	2
2.2. Filosofía de Código e Idioma Institucional	2
3. Estructura Jerárquica del Código Fuente (lib/)	3
4. Punto de Entrada y Ciclo de Vida (lib/main.dart)	4
4.1. Inyección del Árbol de Estado (ProviderScope)	4
4.2. Configuración Temática y Material 3	4
5. Núcleo Transversal del Sistema (lib/nucleo/)	5
5.1. Identidad Cromática Institucional (colores.dart)	5
5.2. Parámetros de Red y Endpoints REST (api-constantes.dart)	5
5.3. Contenedor Monádico Inmutable (resultado.dart)	6
6. Módulo de Autenticación y Gestión de Sesión (lib/modulos/autenticacion/)	6
6.1. Modelo de Usuario y Decodificación Resiliente de JWT (usuario-modelo.dart)	6
6.2. Pantalla de Acceso Perimetral (inicio-sesion-pantalla.dart)	6
6.3. Pantalla de Registro de Becarios (registro-pantalla.dart)	7
6.4. Proveedor Reactivo y Repositorio de Autenticación	7
7. Módulo de Asistencia y Verificación Geoespacial (lib/modulos/asistencia/)	7
7.1. Radar Geofence Concéntrico a 60 FPS (asistencia-pantalla.dart)	7
7.1.1. Características del Radar	8
7.2. Algoritmos Geoespaciales y Protección Anti-Spoofing (asistencia-repositorio.dart)	8
7.3. Confirmación de Ficha Universitaria (confirmar-datos-pantalla.dart)	9
7.4. Tipología de Asistencia (tipo-asistencia.dart)	9
8. Módulo de Viáticos, Pasajes y Recorridos (lib/modulos/pasajes/)	9
8.1. Declaración de Recorridos con Captura Instantánea de PIN GPS (declarar-recorrido-pantalla.dart)	
8.2. Gestión de Estados y Regla Normativa del Día 24 (pasajes-pantalla.dart)	9
8.2.1. Implementación de la Regla del Día 24	10
9. Módulo de Comunicaciones y Notificaciones Push (lib/modulos/notificaciones/)	10
9.1. Tipología y Clasificación de Avisos (notificacion-modelo.dart)	10
9.2. Interacción y Marcado de Lectura (notificaciones-pantalla.dart)	10
10. Módulo de Inicio y Navegación Multitarea (lib/modulos/inicio/)	10
10.1. Preservación del Estado en Memoria (IndexedStack)	11
10.2. Insignia Dinámica Reactiva (Badge)	11
11. Verificación y Suite de Pruebas Automatizadas (test/)	11
12. Guía de Operaciones, Compilación y Mantenimiento	12
12.1. Comandos Operativos Esenciales	12
13. Conclusiones y Certificación Agéntica de Calidad	12

1 Fundamentación y Contexto Operativo

La aplicación móvil **BUMAND** constituye la plataforma táctica y operativa desplegada en dispositivos móviles para los estudiantes pertenecientes al Programa de Becarios Universitarios de la **Fundación Diaconía FRIF-IFD**. Diseñada e implementada sobre el framework de alto rendimiento **Flutter (Dart)**, la aplicación responde a la necesidad crítica de modernizar, transparentar y desmaterializar los procesos tradicionales de supervisión académica, registro de asistencia presencial y rendición de gastos de transporte.

Históricamente, los becarios debían recurrir a formularios impresos en papel para registrar sus horas de práctica preprofesional en las dependencias de la entidad y las sedes eclesiales aliadas. De igual manera, la solicitud de reintegro de viáticos de transporte público (sujeta al reembolso estatutario del 80 %) exigía un llenado manual propenso a inconsistencias aritméticas, pérdidas físicas de boletas y retrasos administrativos considerables. En el contexto geográfico específico de los municipios de La Paz y El Alto —caracterizados por una topografía accidentada y trayectos urbanos complejos—, la ausencia de trazabilidad geoespacial impedía constatar de manera objetiva si el becario se encontraba efectivamente en el recinto universitario asignado o en la sucursal de la institución.

Frente a este escenario, BUMAND Móvil fue concebido como un cliente ligero, altamente reactivo y confiable, capaz de operar bajo condiciones de conectividad intermitente, preservar la sesión del usuario mediante almacenamiento local, verificar la ubicación física del estudiante a través de geocercas poligonales con tolerancia elástica y prevenir de forma proactiva cualquier intento de suplantación digital mediante algoritmos nativos anti-spoofing.

2 Arquitectura de Software y Principios de Diseño

El sistema móvil ha sido edificado bajo los postulados de la **Arquitectura Limpia (Clean Architecture)** adaptada al ecosistema reactivo de Flutter, garantizando un desacoplamiento riguroso entre la lógica de negocio, las fuentes de datos y las capas visuales de presentación.

2.1 Estratificación de Responsabilidades

1. **Capa de Presentación (pantallas/ y widgets-comunes/)**: Compuesta exclusivamente por widgets de Flutter (`StatelessWidget`, `ConsumerStatefulWidget`). No ejecuta peticiones HTTP directas ni procesa lógica de almacenamiento; su única función es proyectar el estado recibido y emitir intenciones de usuario a través de eventos.
2. **Capa de Control de Estado (proveedores/)**: Centralizada mediante la biblioteca **Riverpod** (`flutter_riverpod`). Emplea notificadoros asíncronos (`AsyncNotifier`, `Notifier`) que exponen estados inmutables, gestionan ciclos de vida de carga/error y orquestan la invocación a los repositorios correspondientes.
3. **Capa de Dominio (modelos/)**: Alberga las entidades del negocio, con tipado estricto, constructores constantes, serialización JSON bidireccional y métodos auxiliares de validación semántica.
4. **Capa de Infraestructura y Datos (repositorios/)**: Encapsula el cliente HTTP nativo, la interacción con sensores periféricos de geolocalización (`Geolocator`) y el almacenamiento en disco mediante `SharedPreferences`.

2.2 Filosofía de Código e Idioma Institucional

En estricto cumplimiento de las normativas de ingeniería de software de BUMAND:

- **Español Absoluto:** Todo el código fuente —incluyendo clases, variables, métodos, parámetros, comentarios de hardware opaco y rutas relativas— se encuentra escrito en idioma español sin anglicismos innecesarios.
- **snake_case:** Variables de instancia, propiedades de modelos y atributos de datos (ej. `correo_controlador`, `monto_devolucion`, `es_recordatorio`).
- **camelCase:** Métodos, funciones y constructores de fábrica (ej. `iniciarSesion()`, `obtenerPosicionActual`).
- **PascalCase:** Clases, modelos, enumeradores y Widgets (ej. `UsuarioModelo`, `AsistenciaPantalla`, `TipoAsistencia`).
- **UPPER_SNAKE_CASE:** Constantes globales y tokens cromáticos (ej. `AZUL_DIACONIA`, `RUTA_BASE`).
- **kebab-case:** Nombres de archivos (`.dart`) y directorios de proyecto (ej. `inicio-sesion-pantalla.dart`, `api-constantes.dart`).
- **Invariantes de Rendimiento a 60 FPS:** Uso exhaustivo de constructores `const` y `final` en la jerarquía del árbol de renderizado, evitando reconstrucciones parasitarias de widgets estáticos.

3 Estructura Jerárquica del Código Fuente (lib/)

La distribución modular del código fuente dentro de `bumand-movil/lib/` refleja una segregación limpia por dominios funcionales:

- `main.dart`: Punto de entrada, contenedor raíz `ProviderScope` y configuración de tema Material 3.
- `nucleo/`: Infraestructura transversal compartida.
 - `constantes/api-constantes.dart`: Mapeo de rutas REST y configuración de red.
 - `tema/colores.dart`: Paleta cromática oficial e institucional `BumandColores`.
 - `utilidades/resultado.dart`: Contenedor monádico inmutable `Resultado<T>`.
- `modulos/autenticacion/`: Gestión de identidad y sesión.
 - `modelos/usuario-modelo.dart`: Entidad serializable con decodificador de carga JWT.
 - `pantallas/inicio-sesion-pantalla.dart`: Formulario de acceso perimetral con validación.
 - `pantallas/registro-pantalla.dart`: Formulario de alta con bloqueo de pegado en correo.
 - `proveedores/autenticacion-proveedor.dart`: Estado reactivo global de sesión.
 - `repositorios/autenticacion-repositorio.dart`: Cliente HTTP y persistencia en disco.
- `modulos/asistencia/`: Marcación presencial y verificación geoespacial.
 - `datos/universidades.dart`: Catálogo oficial de 20 universidades de La Paz y El Alto.
 - `modelos/tipo-asistencia.dart`: Enumerador de actividades formativas institucionales.
 - `pantallas/asistencia-pantalla.dart`: Radar concéntrico a 60 fps y control de geocerca.
 - `pantallas/confirmar-datos-pantalla.dart`: Actualización de perfil universitario.
 - `proveedores/asistencia-proveedor.dart`: Estado reactivo de geoposicionamiento y perfil.
 - `repositorios/asistencia-repositorio.dart`: Algoritmo Haversine y anti-spoofing.

- `modulos/pasajes/`: Viáticos, recorridos y regla del 80 %.
 - `modelos/recorrido-modelo.dart`: Modelo de tramo con georreferenciación satelital.
 - `modelos/solicitud-pasaje-modelo.dart`: Solicitud mensual con recorridos anidados.
 - `pantallas/declarar-recorrido-pantalla.dart`: Asistente con botón PIN GPS y cálculo del 80 %.
 - `pantallas/pasajes-pantalla.dart`: Máquina de estados, historial y regla del día 24.
 - `proveedores/pasajes-proveedor.dart`: Notificador asíncrono de solicitudes de pasajes.
 - `repositorios/pasajes-repositorio.dart`: CRUD de tramos y contenedor `ResultadoPasaje`.
 - `utilidades/formato-pasajes.dart`: Utilidad canónica de fechas, monedas y habilitaciones.
- `modulos/notificaciones/`: Sistema de alertas push y recordatorios.
 - `modelos/notificacion-modelo.dart`: Entidad con inferencia tipológica inteligente.
 - `pantallas/notificaciones-pantalla.dart`: Bandeja con chips de filtrado y tarjetas.
 - `proveedores/notificaciones-proveedor.dart`: Notificador reactivo y conteo de no leídas.
 - `repositorios/notificaciones-repositorio.dart`: Cliente HTTP de notificaciones y parche de lectura.
- `modulos/inicio/`: Contenedor principal de navegación.
 - `pantallas/pantalla-inicio.dart`: `NavigationBar` Material 3 con `IndexedStack` y `Badge`.

4 Punto de Entrada y Ciclo de Vida (`lib/main.dart`)

El punto de arranque de la aplicación móvil se instrumenta en `lib/main.dart`:

```
void main() {
  runApp(const ProviderScope(child: BumandApp()));
}
```

4.1 Inyección del Árbol de Estado (`ProviderScope`)

Para habilitar la reactividad en toda la aplicación, `BumandApp` está envuelta en un contenedor `ProviderScope`. Este elemento actúa como el almacén raíz de todos los proveedores Riverpod, asegurando que el estado de autenticación, la información del perfil del becario y la sincronización de las geocercas se mantengan vivos e independientes del ciclo de vida visual de los widgets individuales.

4.2 Configuración Temática y Material 3

La clase `BumandApp` extiende de `StatelessWidget` y declara un `MaterialApp` tipado con las siguientes especificaciones:

- `useMaterial3: true`: Adopta los componentes visuales de última generación de Material Design, incluyendo la elevación tonal, transiciones fluidas y formas elípticas contemporáneas.

- **ColorScheme.fromSeed:** Genera la paleta dinámica del sistema utilizando `BumandColores.NARANJA` como color semilla, estableciendo `BumandColores.NARANJA_OSCURO` como color primario institucional y `BumandColores.AZUL_DIACONIA` como acento secundario.
- **scaffoldBackgroundColor:** `BumandColores.FONDO`: Proporciona un tono de fondo neutro y suave (`#EFF3F8`) que reduce la fatiga visual durante jornadas prolongadas de registro.
- **Ruta Inicial:** Establece a `InicioSesionPantalla` como la puerta de entrada perimetral.

5 Núcleo Transversal del Sistema (lib/nucleo/)

5.1 Identidad Cromática Institucional (colores.dart)

La clase `BumandColores` define los tokens cromáticos inmutables acordes a la memoria de diseño corporativo:

Constante	Código Hex	Muestra	Propósito Operativo Institucional
NARANJA	#FDBC58		Color corporativo semilla, botones primarios e identidad BUMAND.
NARANJA_OSCURO	#E8951F		Enlaces destacados, estados pendientes y contrastes de accesibilidad.
NEGRO	#1A1A1A		Tipografía principal, títulos prominentes y elementos de alto contraste.
AZUL_DIACONIA	#067CC1		Identidad institucional de Diaconía FRIF-IFD, cabeceras y acentos.
TURQUESA_DIACONIA	#1EB5C4		Éxito en geocercas, degradados de cabecera y estados aprobados.
FONDO	#EFF3F8		Lienzo neutro de fondo para vistas y formularios de entrada.
BLANCO	#FFFFFF		Superficie de tarjetas, cajas de texto y contenedores con elevación.

Cuadro 1: Tokens cromáticos oficiales del ecosistema móvil BUMAND.

5.2 Parámetros de Red y Endpoints REST (api-constantes.dart)

La clase `ApiConstantes` centraliza las rutas relativas de la API REST desplegada en NestJS:

- **RUTA_BASE:** Configurada en `http://10.0.2.2:3002` para compatibilidad transparente con el entorno de red de emuladores Android (mapeando a `localhost:3002` en la máquina anfitriona).
- **PUNTO_INICIO_SESION:** `/autenticacion/inicio-sesion` (Emisión de credenciales JWT).
- **PUNTO_REGISTRO:** `/autenticacion/registro` (Alta de nuevos becarios).
- **PUNTO_ASISTENCIA_INGRESO:** `/asistencia/ingreso` (Marcación de entrada sujeta a geocerca).
- **PUNTO_ASISTENCIA_SALIDA:** `/asistencia/salida` (Marcación de salida y cómputo horario).
- **PUNTO_ASISTENCIA_HISTORIAL:** `/asistencia` (Consulta de marcaciones históricas).
- **PUNTO_PASAJES_CONFIRMAR:** `/pasajes/confirmar-datos` (Validación de ficha personal).
- **PUNTO_PASAJES:** `/pasajes` (CRUD de tramos y solicitudes).
- **PUNTO_NOTIFICACIONES:** `/notificaciones/mis-notificaciones` (Bandeja personalizada).
- **PUNTO_NOTIFICACIONES_CONTEO:** `/notificaciones/no-leidas/conteo` (Conteo para insignias).

5.3 Contenedor Monádico Inmutable (resultado.dart)

Para prevenir excepciones no controladas en tiempo de ejecución (*Unhandled Exception*), el sistema implementa el patrón de contenedor monádico funcional `Resultado<T>`:

```
class Resultado<T> {
  final bool exito;
  final T? datos;
  final String? error;

  const Resultado({required this.exito, this.datos, this.error});

  factory Resultado.exitoso(T datos) => Resultado(exito: true, datos: datos);
  factory Resultado.fallido(String error) =>
    Resultado(exito: false, error: error);
}
```

Este patrón asegura que toda llamada a red, procesamiento criptográfico o lectura de sensores retorne un objeto tipado explícito, obligando a las capas de presentación a bifurcar la ejecución entre el éxito de la operación o la visualización contextual del mensaje de fallo.

6 Módulo de Autenticación y Gestión de Sesión (lib/modulos/autenticacion)

6.1 Modelo de Usuario y Decodificación Resiliente de JWT (usuario-modelo.dart)

La entidad `UsuarioModelo` representa la identidad del usuario autenticado. Además de proveer constructores inmutables y mapeo bidireccional JSON en `snake_case`, incorpora un mecanismo interno para decodificar la carga útil del JWT cuando el servidor responde únicamente con el token de acceso:

```
static Map<String, dynamic>? _decodificarJwt(String token) {
  try {
    final partes = token.split('.');
    if (partes.length != 3) return null;
    final normalizado = base64Url.normalize(partes[1]);
    final decodificado = utf8.decode(base64Url.decode(normalizado));
    return jsonDecode(decodificado) as Map<String, dynamic>;
  } catch (_) {
    return null;
  }
}
```

Esta técnica extrae de forma tolerante a fallos los atributos `sub` (identificador primario), `correo` y `rol` directamente de los fragmentos del token `base64Url`, garantizando una carga instantánea de la identidad del usuario sin requerir peticiones de red redundantes.

6.2 Pantalla de Acceso Perimetral (inicio-sesion-pantalla.dart)

La vista `InicioSesionPantalla` implementa una experiencia estética pulida:

- **Campos Estilizados:** Cajas de entrada redondeadas con radio de curvatura de 30px, bordes delgados de color `#E3E9F2` e iconos representativos con fondo translúcido (`AZUL_DIACONIA` al 10%).
- **Visibilidad Conmutable:** Alternador interactivo para visualizar u ocultar la clave alfanumérica.
- **Validación Preventiva:** Impide envíos con campos vacíos mediante retroalimentación local instantánea sin consumir recursos de red.

- **Transición Transparente:** Al autenticarse correctamente, utiliza `Navigator.pushAndRemoveUntil` para enrutar a `PantallaInicio`, destruyendo el historial de navegación para evitar retornos accidentales al login al presionar el botón físico “Atrás”.

6.3 Pantalla de Registro de Becarios (`registro-pantalla.dart`)

Diseñada específicamente para becarios preasignados por la administración:

- **Validación Estricta de CI:** Expresión regular `^\d{5,10}$` que exige el ingreso exclusivo de caracteres numéricos sin extensiones geográficas (ej. “LP” o “SC”).
- **Validación Doble de Correo con Protección Anti-Pegado:** Incorpora dos campos para el correo electrónico y deshabilita de manera expresa la acción del portapapeles (`enableInteractiveSelection: false`) en la casilla de confirmación, forzando la escritura manual para prevenir errores tipográficos humanos que bloqueen la recepción de notificaciones.
- **Longitud Mínima de Contraseña:** Valida que la contraseña cuente con un mínimo de 6 caracteres y coincida exactamente con su verificación.

6.4 Proveedor Reactivo y Repositorio de Autenticación

- **AutenticacionNotificador (`autenticacion-proveedor.dart`):** Gestiona el estado inmutable `AutenticacionEstado(cargando, usuario, error)`. Expone métodos para `iniciarSesion()`, `registrar()`, `limpiarError()`, `verificarSesionExistente()` y `cerrarSesion()`.
- **AutenticacionRepositorio (`autenticacion-repositorio.dart`):** Interactúa directamente con los endpoints del backend NestJS. Tras un inicio de sesión o registro exitoso, serializa y persiste las credenciales en `SharedPreferences` mediante las claves `token`, `usuario_id`, `usuario_nombre`, `usuario_correo` y `usuario_rol`. Permite validar en el arranque de la app si el usuario cuenta con una sesión activa no caducada (`estaAutenticado()`).

7 Módulo de Asistencia y Verificación Geoespacial (`lib/modulos/asistencia`)

7.1 Radar Geofence Concéntrico a 60 FPS (`asistencia-pantalla.dart`)

La pantalla de asistencia destaca por su indicador visual de radar en tiempo real, implementado mediante un `AnimationController` con duración de 2,200 ms y un `CustomPainter` dedicado (`_RadarPintor`):

```
class _RadarPintor extends CustomPainter {
  final double progreso;
  final Color color_radar;
  final bool activo;

  @override
  void paint(Canvas lienzo, Size tamano) {
    final centro = Offset(tamano.width / 2, tamano.height / 2);
    final radio_maximo = tamano.shortestSide / 2;

    for (int i = 0; i < 3; i++) {
      final fase = (progreso + (i * 0.33)) % 1.0;
      final radio = 26.0 + (fase * (radio_maximo - 26.0));
      final opacidad = activo ? ((1.0 - fase) * 0.45).clamp(0.0, 1.0) : 0.12;

      final pintura_relleno = Paint()
        ..color = color_radar.withValues(alpha: opacidad * 0.2)
        ..style = PaintingStyle.fill;
    }
  }
}
```



```

final pintura_borde = Paint()
  ..color = color_radar.withValues(alpha: opacidad)
  ..style = PaintingStyle.stroke
  ..strokeWidth = 2.0;

lienzo.drawCircle(centro, radio, pintura_relleno);
lienzo.drawCircle(centro, radio, pintura_borde);
}
}
}

```

7.1.1 Características del Radar

- **Tres Anillos de Onda Concéntricos:** Se generan dinámicamente con un desfase armónico de un tercio de ciclo ($i \times 0.33$), expandiéndose desde el núcleo central de 26 px hasta el borde exterior.
- **Desvanecimiento Progresivo:** La opacidad de cada anillo decrece exponencialmente a medida que su radio se aproxima al límite máximo, generando un efecto visual continuo de sonar.
- **Semáforo Cromático de Estados:**
 - *Azul Diaconía (#067CC1):* Modo de escaneo activo con icono de radar giratorio.
 - *Verde Esmeralda (#10B981):* Usuario posicionado legítimamente dentro de la geocerca permitida.
 - *Rojo Alerta (#EF4444):* Usuario fuera de rango; marcación estrictamente inhabilitada.

7.2 Algoritmos Geoespaciales y Protección Anti-Spoofing (asistencia-repositorio.dart)

Para asegurar la validez jurídica e institucional de los registros:

1. Detección de Ubicaciones Simuladas (Anti-Spoofing):

```
if (posicion.isMocked) return false;
```

Si el sistema operativo detecta que las coordenadas provienen de una aplicación de simulación de GPS (*Fake GPS*) o un emulador de hardware, el método invalida inmediatamente la geocerca.

2. **Fórmula de Haversine y Margen de Tolerancia Elástica:** Calcula la distancia esférica sobre el geoide terrestre entre la posición actual del dispositivo y el vértice central asignado al lugar de práctica:

$$\text{Margen Dinámico} = \text{clamp}(\text{posicion.accuracy}, 0.0, 10.0) \quad (1)$$

$$\text{Distancia} \leq \text{Radio de Tolerancia} + \text{Margen Dinámico} \quad (2)$$

Este margen compensa las fluctuaciones electromagnéticas menores del satélite sin permitir fraudes deliberados.

3. **Cálculo de Precisión de Marcación:** Se transmiten al backend la latitud, longitud, nivel de precisión métrica (*accuracy*), estado de simulación (*isMocked*) y el tipo de jornada seleccionada.

7.3 Confirmación de Ficha Universitaria (`confirmar-datos-pantalla.dart`)

Permite a los becarios verificar y actualizar su institución universitaria antes de procesar trámites de transporte. Ofrece un selector desplegable basado en el catálogo institucional UNIVERSIDADES.LA PAZ.EL ALTO (`universidades.dart`) con cobertura exhaustiva de universidades públicas, privadas y de régimen especial (UMSA, UPEA, UCB, EMI, UPB, UNIVALLE, UNIFRANZ, etc.), disponiendo adicionalmente de un campo abierto para ingresar instituciones no listadas.

7.4 Tipología de Asistencia (`tipo-asistencia.dart`)

El enumerador `TipoAsistencia` tipifica los tres pilares de formación del programa:

1. `practicass`: “Registro de Prácticas” (desempeño técnico institucional en dependencias de Diaconía).
2. `escuelaLideres`: “Escuela de Líderes” (capacitación y mentoría sabatina).
3. `accionesServicio`: “Acciones de Servicio” (voluntariado y liderazgo social diaconal).

8 Módulo de Viáticos, Pasajes y Recorridos (`lib/modulos/pasajes/`)

8.1 Declaración de Recorridos con Captura Instantánea de PIN GPS (`declarar-recorrido-p`)

Esta pantalla permite registrar tramos de traslado con altos niveles de usabilidad:

- **Botón PIN GPS (Captura Satelital)**: Tanto para el punto de origen como para el de destino, el usuario puede presionar el botón de pin para capturar con un solo toque sus coordenadas geográficas actuales con indicación de error de precisión (ej. ± 8 m), asociándolas de forma transparente al tramo declarado.
- **Inversión Automática de Tramos (Ida y Vuelta)**: Mediante un conmutador (`SwitchListTile`), la aplicación permite registrar en un solo paso el trayecto de regreso, invirtiendo de forma instantánea el origen y destino, intercambiando las posiciones satelitales y solicitando únicamente la tarifa de retorno.
- **Persistencia en Memoria Local**: Los puntos de origen, destino y tarifas habituales se conservan en `SharedPreferences` (`pasajes_ultimo_origen`, `pasajes_ultimo_destino`), evitando la tediosa recaptura manual de trayectos rutinarios.
- **Cálculo en Tiempo Real del 80 %**: El componente `_ResumenDevolucion` proyecta al instante el total del día y la devolución exacta esperada calculada mediante aritmética en centavos enteros:

$$\text{Devolución (80 \%)} = \frac{\text{round}(\text{Total} \times 0.8 \times 100)}{100} \quad (3)$$

8.2 Gestión de Estados y Regla Normativa del Día 24 (`pasajes-pantalla.dart`)

El historial mensual organiza las solicitudes bajo una máquina de estados determinista:

- **borrador (En preparación)**: Permite agregar nuevos tramos o eliminar tramos individuales mediante `_eliminarRecorrido()`.
- **pendiente (En revisión)**: Formulario bloqueado para edición, a la espera de la firma electrónica o validación del mentor.
- **aprobado (Aprobado)**: Formulario certificado para liquidación financiera.
- **rechazado (Con observaciones)**: Devuelto por el supervisor con retroalimentación explícita obligatoria; permite al becario corregir o agregar tramos y volver a enviar.

8.2.1 Implementación de la Regla del Día 24

Conforme a los reglamentos contables de Diaconía, las solicitudes mensuales no pueden enviarse formalmente hasta el **día 24** del mes en curso, asegurando que el estudiante consolide la totalidad de sus viajes del periodo antes del cierre administrativo:

```
static bool puedeEnviar(String periodo_str) {
  final hoy = DateTime.now();
  return !DateTime(
    hoy.year,
    hoy.month,
    hoy.day,
  ).isBefore(fechaHabilitada(periodo_str));
}
```

Si el becario intenta enviar el formulario antes de la fecha límite, el botón permanece inhabilitado y se proyecta una leyenda aclaratoria que especifica la fecha exacta de apertura institucional.

9 Módulo de Comunicaciones y Notificaciones Push (lib/modulos/notificac

9.1 Tipología y Clasificación de Avisos (notificacion-modelo.dart)

El modelo `NotificacionModelo` clasifica semánticamente cada alerta entrante para enriquecer la experiencia visual del usuario:

- **Recordatorios de Salida (esRecordatorioSalida):** Notifica al becario si olvidó registrar su marcación de egreso al transcurrir el horario previsto de su práctica. Se representa con icono de reloj de alarma (`Icons.alarm_on_rounded`) y paleta azul cielo (`#E0F2FE` / `#067CC1`).
- **Pasajes Observados (esPasajeObservado):** Comunica que una solicitud mensual de viáticos fue rechazada o devuelta para subsanación. Se destaca con icono de advertencia (`Icons.warning_amber_rounded`) y paleta ámbar institucional (`#FEF3C7` / `#E8951F`).
- **Evaluaciones Académicas (esEvaluacion):** Avisos sobre habilitación o publicación de calificaciones de la Matriz 360° o Formulario F-03. Se estiliza en tonalidad violeta (`#F3E8FF` / `#7C3AED`).
- **Avisos Generales:** Comunicados institucionales ordinarios en gris pizarra.

9.2 Interacción y Marcado de Lectura (notificaciones-pantalla.dart)

- **Filtros Segmentados:** Barra horizontal de `ChoiceChips` que permite conmutar rápidamente entre “Todas”, “Recordatorios”, “Pasajes” y “Evaluaciones”.
- **Marcado Táctil Asíncrono:** Al tocar cualquier tarjeta no leída (identificada por un punto cromático indicador), el sistema despacha una solicitud `PATCH /notificaciones/:id/leer` a través de `NotificacionesNotificador.marcasComoLeida()`, actualizando inmediatamente la interfaz visual y decrementando el contador global.

10 Módulo de Inicio y Navegación Multitarea (lib/modulos/inicio/)

La experiencia de navegación global de la aplicación se encuentra centralizada en `PantallaInicio` (`pantalla-inicio.dart`).

10.1 Preservación del Estado en Memoria (IndexedStack)

A diferencia de los enfoques convencionales que destruyen y recrean los árboles de widgets al cambiar de pestaña en el menú inferior, BUMAND Móvil emplea un `IndexedStack`:

- **Cero Pérdida de Datos en Formularios:** Si un becario se encuentra capturando un recorrido largo en la pestaña de pasajes y conmuta temporalmente a la pestaña de asistencia para verificar su geocerca, al retornar a pasajes todos los campos de texto, fechas y pines GPS se mantienen intactos.
- **Eficiencia Gráfica:** Mantiene los controladores de animación del radar de geocerca en estado de reposo mientras la pestaña correspondiente no está en foco primario, optimizando el consumo de batería del terminal móvil.

10.2 Insignia Dinámica Reactiva (Badge)

La tercera pestaña de la barra inferior (Notificaciones) incorpora un widget `Badge` reactivo conectado a `notificaciones_conteo_proveedor`. Cuando existen alertas sin leer, la insignia muestra el número entero exacto superpuesto sobre el icono de campana, actualizándose instantáneamente sin requerir recargas manuales.

11 Verificación y Suite de Pruebas Automatizadas (test/)

La robustez de la aplicación móvil está respaldada por una exhaustiva suite de **39 pruebas automatizadas** (unitarias y de widgets), las cuales reportan una tasa de éxito del **100 %** al ejecutarse con `flutter test`.

Archivo de Prueba	Casos	Éxito	Tipo	Comportamiento Validado
<code>widget.test.dart</code>	1	100 %	Humo	Inicialización de <code>BumandApp</code> y montaje del árbol.
<code>resultado.test.dart</code>	2	100 %	Unitario	Contenedor <code>Resultado<T></code> (éxito y manejo de error).
<code>usuario-modelo.test.dart</code>	5	100 %	Unitario	Serialización JSON, deserialización y parsing JWT.
<code>inicio-sesion-pantalla.test.dart</code>	5	100 %	Widget	Validación, errores HTTP 401, loaders y navegación.
<code>autenticacion-proveedor.test.dart</code>	5	100 %	Estado	Ciclo de vida de <code>AutenticacionNotificador</code> .
<code>autenticacion-repositorio.test.dart</code>	6	100 %	Mock HTTP	Validación previa, persistencia en disco y registro.
<code>asistencia-pantalla.test.dart</code>	1	100 %	Widget	Renderizado del radar geocerca y botones condicionales.
<code>asistencia-repositorio.test.dart</code>	4	100 %	Algoritmo	Haversine, margen elástico y rechazo anti-spoofing.
<code>declarar-recorrido-pantalla.test.dart</code>	1	100 %	Widget	Formulario de viáticos y botones de PIN GPS.
<code>notificacion-modelo.test.dart</code>	5	100 %	Unitario	Tipología semántica y serialización bidireccional.
<code>notificaciones-pantalla.test.dart</code>	4	100 %	Widget	Filtros horizontales, estado vacío y lectura interactiva.
Total Acumulado	39	100 %	39 pruebas aprobadas en 1.5 s con cero advertencias	

12 Guía de Operaciones, Compilación y Mantenimiento

12.1 Comandos Operativos Esenciales

- **Obtención de Dependencias:**

```
flutter pub get
```

- **Análisis Estático de Código:**

```
flutter analyze
```

- **Ejecución de Suite de Pruebas:**

```
flutter test
```

- **Compilación de Producción Android APK / App Bundle:**

```
flutter build apk --release
flutter build appbundle --release
```

- **Ejecución Local Multiplataforma:**

```
flutter run                # Emulador o dispositivo fisico Android / iOS
flutter run -d windows     # Entorno nativo Windows Desktop
flutter run -d chrome      # Navegador web Google Chrome
```

13 Conclusiones y Certificación Agéntica de Calidad

La aplicación móvil **BUMAND Móvil** consolida un estándar de ingeniería móvil de máxima categoría para la **Fundación Diaconía FRIF-IFD**:

1. **Rigor Arquitectónico:** La adopción de Clean Architecture y Riverpod erradica dependencias cruzadas entre la lógica de negocio y los widgets, facilitando el mantenimiento a largo plazo y la extensibilidad del sistema.
2. **Seguridad Geoespacial y Financiera:** La implementación de algoritmos matemáticos estrictos en centavos para la regla del 80% y la verificación de geocercas con protección nativa contra ubicaciones simuladas (`isMocked`) garantizan la probidad en el desembolso de fondos institucionales.
3. **Calidad Verificada al 100 %:** Con 39 pruebas automatizadas aprobadas y cero advertencias en el análisis estático de Dart, el código fuente entrega una base estable, rápida y lista para producción.

Supervisión Tecnológica
Fundación Diaconía FRIF-IFD

Dirección de Programa Diaconal
Proyecto BUMAND